a02_3

April 22, 2026

Anh Tu Duong Nguyen (anguyea, 115931)

Anh-Nhat Nguyen (anhnnguy, 2034311)

[10]:
```
import numpy as np
import torch

from a02_functions import SimpleCNN, train_model
from a02_helper import get_raw_data, tsne, tsne_plot, DEVICE, examples_heatmap,␣
↪nextplot

%load_ext autoreload
%autoreload 2
```

```
The autoreload extension is already loaded. To reload it, use:
  %reload_ext autoreload
```

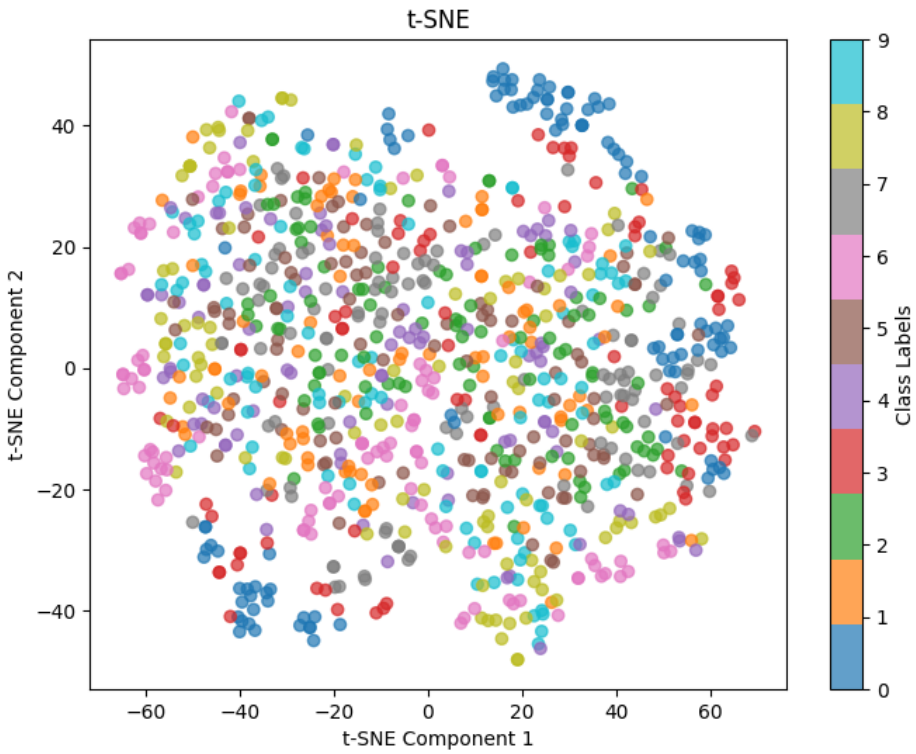
1 Task 3: Visualization

1.0.1 t-SNE for input data

[11]:
```
perplexity = 30
n_examples = 1000  # use lower number for faster computation

np.random.seed(0)
rand_indices = np.random.choice(4000, min(4000, n_examples))

data = get_raw_data()
x = data["x"][rand_indices]
y = data["y"][rand_indices]
x_test = data["x_test"]
y_test = data["y_test"]
```

[12]:
```
x_tsne = tsne(x, perplexity=perplexity)
```

[13]:
```
nextplot()
tsne_plot(x_tsne, y)
```



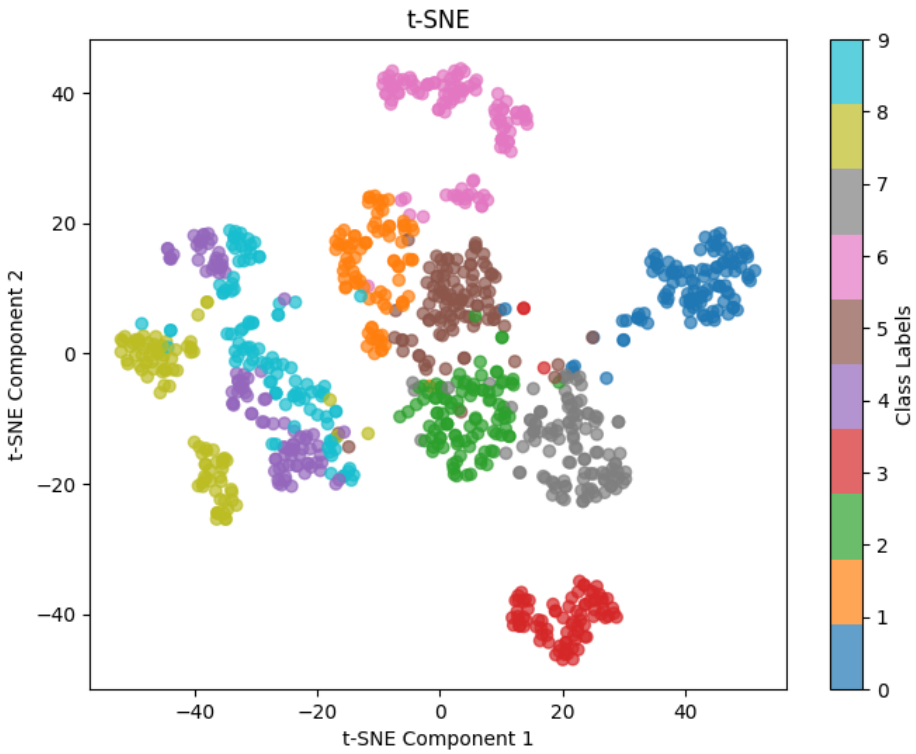
1.0.2 t-SNE for embeddings of training data

[14]:
```
torch.manual_seed(0)
cnn = SimpleCNN()
_ = train_model(data, cnn)
```

```
epoch 0, dt 0.28s, train_loss 1.515e+00, val_loss 1.446e+00, train_acc 40.7,
val_acc 40.8
epoch 10, dt 1.87s, train_loss 4.498e-01, val_loss 7.026e-01, train_acc 80.0,
val_acc 70.4
epoch 20, dt 1.86s, train_loss 2.554e-01, val_loss 4.688e-01, train_acc 93.4,
val_acc 85.8
epoch 30, dt 2.27s, train_loss 5.536e-02, val_loss 7.090e-01, train_acc 95.5,
val_acc 84.8
epoch 40, dt 1.84s, train_loss 4.144e-02, val_loss 6.942e-01, train_acc 98.2,
val_acc 87.9
epoch 50, dt 1.83s, train_loss 3.477e-02, val_loss 6.134e-01, train_acc 99.3,
val_acc 89.2
epoch 60, dt 2.13s, train_loss 8.532e-02, val_loss 7.925e-01, train_acc 98.0,
val_acc 88.5
epoch 70, dt 2.33s, train_loss 4.603e-03, val_loss 6.392e-01, train_acc 100.0,
```

```
val_acc 88.8
epoch 80, dt 2.07s, train_loss 4.246e-04, val_loss 6.745e-01, train_acc 100.0,
val_acc 90.0
epoch 90, dt 2.58s, train_loss 5.494e-04, val_loss 7.000e-01, train_acc 100.0,
val_acc 90.5
Final result: train_loss 3.064e-04, val_loss 7.000e-01, test_loss 5.889e-01,
train_acc 100.0, val_acc 90.5, test_acc 92.2
```

[15]:
```
# Before you start, populate `embedding` during the forward pass (in SimpleCNN).
 ↪ It
# should be a list with one element per layer, each being a tensor of form␣
 ↪examples x
# embeddings.
cnn.store_embeddings = True
with torch.no_grad():
    cnn(torch.Tensor(x).to(DEVICE))
train_embeddings = [e.cpu().numpy() for e in cnn.embeddings]

# TODO: your code here
# Task 3b+d: t-SNE on training embeddings after last conv-block (i.e., before␣
 ↪the linear prediction head)
emb_train = train_embeddings[-1].reshape(len(train_embeddings[-1]), -1)
train_tsne = tsne(emb_train, perplexity=perplexity)
nextplot()
tsne_plot(train_tsne, y)
```



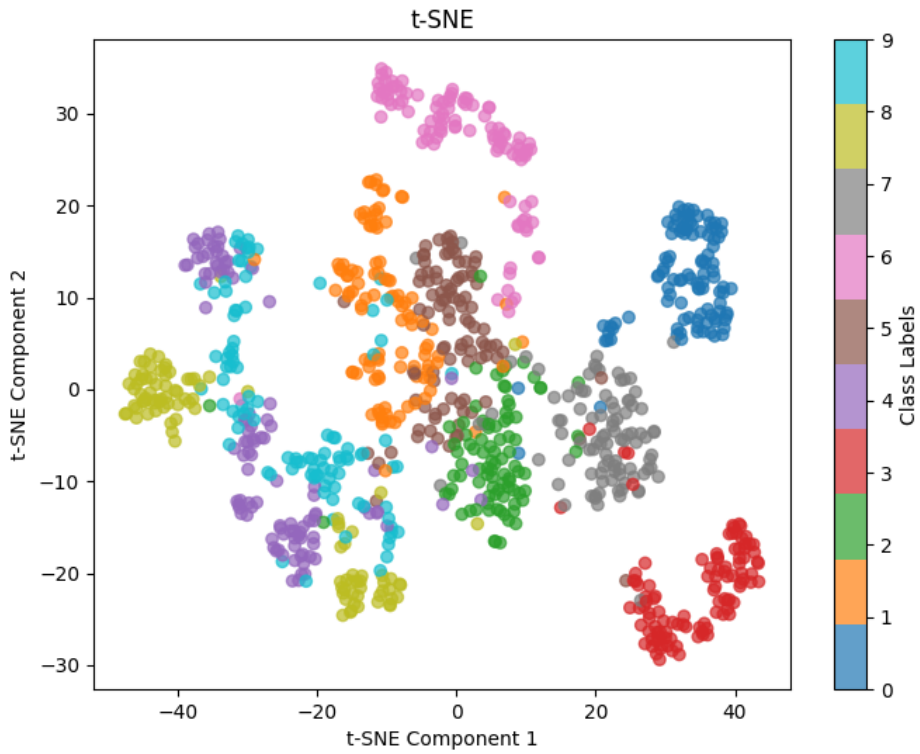
[ ]:

```

```

1.0.3 t-SNE for embeddings of test data

[16]:

```
# TODO: your code here
# Task 3b+d: extract test embeddings and plot t-SNE
cnn.store_embeddings = True
with torch.no_grad():
    cnn(torch.Tensor(x_test).to(DEVICE))
test_embeddings = [e.cpu().numpy() for e in cnn.embeddings]

emb_test = test_embeddings[-1].reshape(len(test_embeddings[-1]), -1)
test_tsne = tsne(emb_test, perplexity=perplexity)
nextplot()
tsne_plot(test_tsne, y_test)
```



1.0.4 Embeddings Heatmap

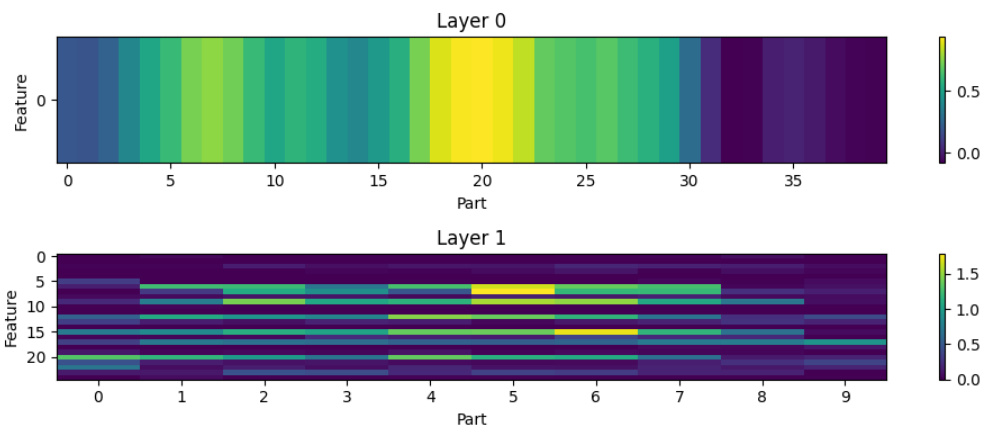
Following the example below, create a plot that visualizes the average activation strength over all examples from a certain class `cls` for all features (channels).

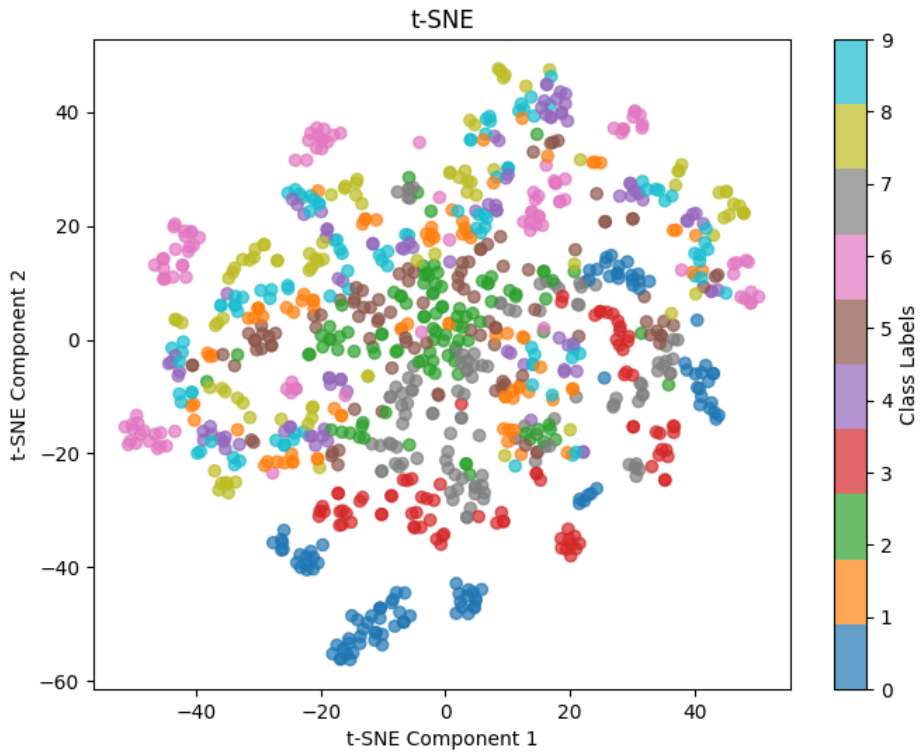
It might help to take a look into the plot function `embedding_heatmap`. Let `cls = 0` for instance, i.e., let us start with the "digit" 0. The dataset (depending on the seed and generation parameters) contains 419 examples for that digit (approximately 10% of all training examples). Since the length of each "digit" is 40, i.e., 40 *parts*, the input dimensionality is $(419, 1, 40)$. Each example is one dimensional, i.e., we only have single feature per example. In the first heatmap (*Input*), we average over all examples and hence see the average value of the only feature (y-axis) for the individual parts (for each "time step"; x-axis).

After first layer in your CNN (when `stride = 2` and `channels = 25`), the dimensionality of its output is $(419, 25, 20)$, so the number of parts has halved due to the stride, but there are now 25 features due to the number of channels.

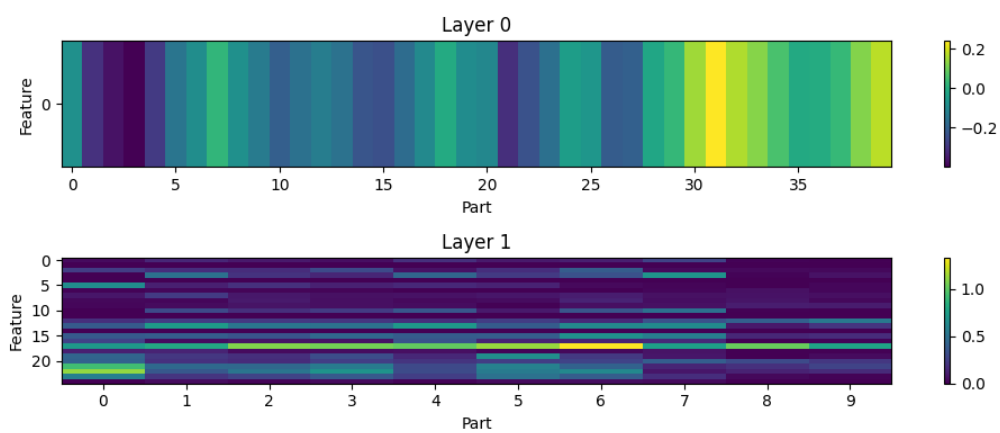
Now, in the heatmap below, we average over all examples of class `cls`. This means that the heatmap shows the average activations over all examples of that class `cls` for all features (y-axis) and parts (x-axis).

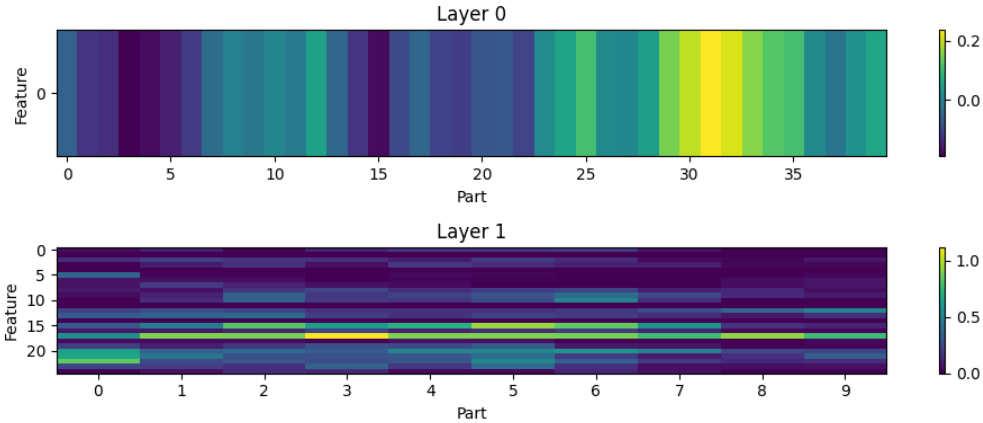
[17]:
```
cnn.store_embeddings = True
with torch.no_grad():
    cnn(torch.Tensor(x).to(DEVICE))

train_embeddings = (
    cnn.embeddings
)  # a list of per-layer embeddings, each of form example x embedding

cls = 0  # investigate different classes by changing this index
cls_examples = torch.Tensor(x[y == cls])
cls_train_embeddings = [e[y == cls] for e in train_embeddings]

nextplot()
examples_heatmap([cls_examples] + cls_train_embeddings[:-1])

# TODO: your code here
# Task 3e: t-SNE using conv-block 1 embeddings (earlier layer) for comparison
emb_train_block1 = train_embeddings[0].reshape(len(train_embeddings[0]), -1)
train_tsne_block1 = tsne(emb_train_block1, perplexity=perplexity)
nextplot()
tsne_plot(train_tsne_block1, y)
```





[18]:
```
# Task 3f: heatmap for a different class to compare feature activations
for cls in [1, 5]:
    cls_examples_other = torch.Tensor(x[y == cls])
    cls_embeddings_other = [e[y == cls] for e in train_embeddings]
    nextplot()
    examples_heatmap([cls_examples_other] + cls_embeddings_other[:-1])
```




Layer 0
Feature
0
0
5
10
15
20
25
30
35
Part
0.2
0.0
Layer 1
Feature
0
5
10
15
20
0
1
2
3
4
5
6
7
8
9
Part
1.0
0.5
0.0

```
[ ]:
```